Mini-project 1 in Advanced Machine Learning (02460)

by Bashar Bdewi (s183356) on 05 March 2026

Part A: Priors for VAEs (binarized MNIST)

We trained variational autoencoders (VAEs) [1] with Bernoulli likelihood $p_\theta(x \mid z)$ on binarized MNIST. We compared three priors: standard Gaussian, mixture of Gaussians (MoG), and a flow-based prior (RealNVP-style affine coupling) [4]. Encoder/decoder use MLPs with two 512-unit hidden layers; the encoder outputs $(\mu, \log\sigma)$ for $q_\phi(z \mid x)$ and the decoder outputs Bernoulli logits for $p_\theta(x \mid z)$. We use a 2D latent space for visualization. For evaluation we report the test-set log-likelihood approximated by the ELBO lower bound

$$\mathrm{ELBO}(x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \mathrm{KL}(q_\phi(z \mid x) \| p(z)). \tag{1}$$

Table 1: Test ELBO (mean±std over 5 runs; higher is better).

Prior	ELBO mean	ELBO std
Gaussian	−135.78	0.80
MoG	−136.61	0.49
Flow	−135.60	1.01

The Gaussian and flow priors achieved similar ELBO, while MoG was slightly worse. To assess how well the prior matches the learned latent structure, we compare samples from the prior $p(z)$ with samples from the aggregate posterior $q(z) = \frac{1}{N}\sum_i q_\phi(z \mid x_i)$ in Fig. 1. The flow prior can represent a more complex latent distribution than the fixed Gaussian; the MoG prior adds multi-modality, but still shows mismatch in low-density regions.

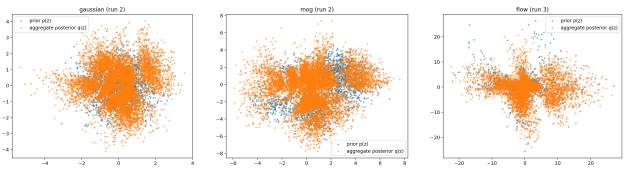

Figure 1: Prior vs aggregate posterior in 2D latent space (Gaussian run 2, MoG run 2, Flow run 3).

Part B: Sampling quality (standard MNIST)

We trained (i) an image-space DDPM [2] with a U-Net denoiser and (ii) a latent DDPM operating in the latent space of a β-VAE [3] with Gaussian likelihood. The image DDPM uses a small U-Net denoiser, while the latent DDPM uses a fully-connected MLP (FcNetwork) operating on 16-D latents. The VAE of choice is the β-VAE with $\beta = 10^{-4}$. Fig. 2 shows four representative samples from each model.

Figure 2: Four samples each: DDPM (left), latent DDPM ($\beta = 10^{-6}$, middle), and β-VAE ($\beta = 10^{-4}$, right).

We quantified sample quality using the Fréchet Inception Distance (FID) [5] (lower is better) and measured sampling speed (samples/second). Results are shown in Table 2.

Table 2: FID and sampling speed (2000 samples).

Model	FID ↓	Samples/s ↑
DDPM (image)	437.44	2.95
Latent DDPM ($\beta = 10^{-6}$)	79.00	1.21×10^4
Latent DDPM ($\beta = 10^{-4}$)	81.04	1.09×10^4
Latent DDPM ($\beta = 10^{-2}$)	80.51	1.16×10^4
β-VAE choice ($\beta = 10^{-4}$)	186.67	1.01×10^5

The image-space DDPM gives the slowest sampling (iterative denoising over $T = 200$ steps) and poor FID under our limited training budget, while the latent DDPM achieves much better FID with orders-of-magnitude faster sampling due to the low-dimensional latent space. The VAE is fastest (single forward pass) but has higher FID than latent DDPM. Fig. 3 indicates that the latent DDPM samples overlap more with the aggregate posterior than the VAE prior.

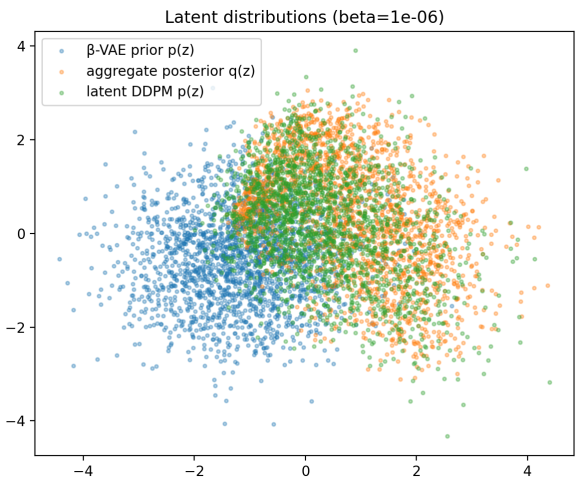

Figure 3: Latent distributions for $\beta = 10^{-6}$: VAE prior $p(z)$, aggregate posterior $q(z)$, and learned latent DDPM distribution.

Code snippets

(1) ELBO for a non-Gaussian prior (MoG/Flow):

```
# KL(q||p) = E_q[ log q(z|x) - log p(z) ] for non-Gaussian priors
q = encoder(x)
z = q.rsample()
recon = decoder(z).log_prob(x)
kl_est = q.log_prob(z) - prior.log_prob(z)
elbo = (recon - kl_est).mean()
loss = -elbo
```

(2) Central parts of the Flow (affine coupling + log-prob):

```
# affine coupling layer forward
z_prime = mask*x + (1-mask)*(x*exp(s(mask*x)) + t(mask*x))
log_det = ((1-mask)*s(mask*z_prime)).sum(-1)

# flow log_prob via change-of-variables
z, log_det = flow.inverse(x)
logp = base().log_prob(z) + log_det
```

(3) DDPM ELBO / training loss (noise-prediction):

```
# sample t ~ Uniform{1,...,T}
# x_t = sqrt(alpha_bar_t)*x0 + sqrt(1-alpha_bar_t)*eps
# minimize MSE between predicted and true noise
eps = torch.randn_like(x0)
xt = sqrt_ab[t]*x0 + sqrt_1mab[t]*eps
pred = net(xt, t)
loss = F.mse_loss(pred, eps)
```

(4) DDPM sampling algorithm (reverse diffusion):

```
# start from x_T ~ N(0, I)
x = torch.randn(batch, dim)
for t in reversed(range(1, T+1)):
    eps_hat = net(x, t)
    mean = (1/sqrt_a[t])*(x - (beta[t]/sqrt_1mab[t])*eps_hat)
    x = mean + sigma[t]*torch.randn_like(x)  # noise except at t=1
```

(5) Latent DDPM training + decoding samples:

```
# build latent dataset z ~ q_phi(z|x) from trained beta-VAE
Z = []
for x,_ in train_loader:
    z = vae.encoder(x).sample()
    Z.append(z.cpu())
Z = torch.cat(Z)

# train DDPM in latent space
train_ddpm(ddpm_z, opt_z, [(z,None) for (z,) in latent_loader], ...)

# sample latents and decode to images
z_gen = ddpm_z.sample((N, latent_dim))
x_gen = vae.decoder(z_gen).mean.clamp(0,1)
```

Declaration of use of generative AI

- I have used generative AI tools: **yes**

Tools used:

- ChatGPT (OpenAI) — used for debugging assistance and code organization.

How the tools were used:

What did you use the tool(s) for? Understanding project requirements, and implementing plotting utilities.

At what stage(s) of the process did you use the tool(s)? During implementation and during report writing.

How did you use or incorporate the generated output? Suggestions were reviewed, adapted, and tested. All final design decisions, experiments, and interpretations are my own responsibility.

References

[1] D. P. Kingma and M. Welling. Auto-Encoding Variational Bayes. In ICLR, 2014.

[2] J. Ho, A. Jain, and P. Abbeel. Denoising Diffusion Probabilistic Models. In NeurIPS, 2020.

[3] I. Higgins et al. β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. In ICLR, 2017.

[4] L. Dinh, J. Sohl-Dickstein, and S. Bengio. Density Estimation using Real NVP. In ICLR, 2017.

[5] M. Heusel et al. GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. In NeurIPS, 2017.